



公开

Orbbec OpenNI2.0-SDK C++接口使用规范

密级

公开

修订历史

版本	修订者	日期	
0.9.0	Zhonghong	2019/04/23	新建初版
0.9.1	Zhonghong	2019/05/31	增加 Openni 常用接口介绍
0.9.2	Zhonghong	2019/06/06	增加 Recorder 录像接口介绍
0.9.3	Lixiaobin	2019/07/04	文档整理和部分修改
0.9.4	zhonghong	2019/7/11	增加 3.2.12 CameraSetting 类
0.9.5	zhonghong	2019/7/15	增加泛光灯控制接口 3.3.16 和 3.3.17
0.9.6	zhonghong	2019/9/5	1. 增加双目切换左右 IR 的接口 3.3.18 2. 修改 3.1.3.2 纯净 IR 在 create 流后,start 流前设置参数;
0.9.7	zhonghong	2019/11/2	1. 增加 6.10 设置 IR 输出格式; 2. 3.2.12.2 setAutoExposureEnabled 函数 修改
0.9.8	Lixiaobin	2021/08/26	新增 IR Face AE roi 接口
0.9.9	Lixiaobin	2021/12/17	修改个接口使用规范
1.0.0	Lixiaobin	2022/01/24	增加右 IR 流创建以及输出
1.0.1	Lixiaobin	2022/04/01	增加设置指定相机标定参数（仅个别模组支持）
1.0.2	Lixiaobin	2022/05/25	增加“3.3.5 获取哈雷相机保存标定参数”接口



目录

修订历史	1
1 简述	5
2 openni2.0 架构	5
3 Openni2 对外的接口	6
3.1 视频数据接口	6
3.1.1 主动获取	6
3.1.2 被动获取	9
3.1.3 获取 IR	10
3.1.3.1 散斑 IR	10
3.1.3.2 右散斑 IR	11
3.1.3.3 纯净 IR	11
3.1.3.4 纯净 IR 切换散斑 IR	12
3.1.4 获取 Color (openni)	13
3.2 Openni 常用接口介绍	13
3.2.1 Device 类	13
3.2.1.1 open 函数	13
3.2.1.2 close() 函数	13
3.2.1.3 setDepthColorSyncEnabled 函数	14
3.2.1.4 setImageRegistrationMode 函数	14
3.2.1.5 setProperty 函数[1]	14
3.2.1.6 setProperty 函数[2]	14
3.2.1.7 getProperty 函数[1]	15
3.2.1.8 getProperty 函数[2]	15
3.2.1.9 getDeviceInfo 函数	15
3.2.2 DeviceConnectedListener 类	15
3.2.3 DeviceDisconnectedListener 类	15
3.2.4 DeviceInfo 类	16
3.2.5 DeviceStateChangedListener 类	16
3.2.6 NewFrameListener 类	16
3.2.7 OpenNI 类	16
3.2.7.1 initialize 函数	16
3.2.7.2 shutdown 函数	17
3.2.7.3 waitForAnyStream 函数	17
3.2.7.4 getVersion 函数	17
3.2.7.5 enumerateDevices 函数	17
3.2.7.6 addDeviceConnectedListener 函数	17
3.2.7.7 removeDeviceConnectedListener 函数	18
3.2.7.8 addDeviceDisconnectedListene 函数	18
3.2.7.9 removeDeviceDisconnectedListener 函数	18
3.2.7.10 addDeviceStateChangedListener 函数	18
3.2.7.10 removeDeviceStateChangedListener 函数	19
3.2.8 VideoFrameRef 类	19



3.2.8.1	getData 函数.....	19
3.2.8.2	getDataSize 函数	19
3.2.8.2	getHeight 函数	19
3.2.8.3	getWidth 函数.....	19
3.2.8.4	getTimestamp 函数.....	19
3.2.8.5	getVideoMode 函数.....	20
3.2.9	VideoMode 类.....	20
3.2.9.1	setPixelFormat 函数	20
3.2.9.2	setResolution 函数	20
3.2.10	VideoStream 类.....	21
3.2.10.1	addNewFrameListener 函数.....	21
3.2.10.2	removeNewFrameListener 函数.....	21
3.2.10.3	create 函数	21
3.2.10.4	start 函数.....	22
3.2.10.5	stop 函数	22
3.2.10.6	setVideoMode 函数	22
3.2.10.7	getVideoMode 函数.....	22
3.2.10.8	readFrame 函数.....	22
3.2.10.9	setProperty 函数[1].....	23
3.2.10.10	setProperty 函数[2].....	23
3.2.10.11	getProperty 函数[1].....	23
3.2.10.12	getProperty 函数[2].....	23
3.2.11	Recorder 类	24
3.2.11.1	attach 函数	24
3.2.11.2	create 函数	24
3.2.11.3	start 函数.....	24
3.2.11.4	stop 函数	24
3.2.11.5	destroy 函数	24
3.2.12	CameraSettings 类.....	25
3.2.12.1	getAutoExposureEnabled 函数	25
3.2.12.2	setAutoExposureEnabled 函数.....	25
3.2.12.3	setGain 函数.....	25
3.2.12.4	setExposure 函数	25
3.2.12.5	getGain 函数.....	26
3.2.12.5	getExposure 函数	26
3.2.11.6	录像例子	26
3.3	扩展 Api 接口.....	28
3.3.1	获取固件版本号	28
3.3.2	获取设备序列号.....	28
3.3.3	获取设备类型	28
3.3.4	获取相机保存标定参数	29
3.3.5	获取哈雷相机保存标定参数	29
3.3.6	获取 IR 增益.....	31
3.3.7	设置 IR 增益.....	32



3.3.8	获取 IR 曝光.....	32
3.3.9	设置 IR 曝光.....	32
3.3.10	设置 LDP 开关 1.....	32
3.3.11	设置 LDP 开关 2.....	33 32
3.3.12	获取 LDP 使能状态.....	33
3.3.13	设置 LDM 激光开关.....	33
3.3.14	设置 LDM 激光开关.....	33
3.3.15	获取 LDM 激光开关状态.....	34 33
3.3.16	设置泛光灯开关.....	34
3.3.17	获取泛光灯开关状态.....	34
3.3.18	切换左右 IR.....	34
3.3.18	设置 IR FaceAEROI.....	34
3.3.19	设置指定相机标定参数.....	35
4	硬件 D2C	35
5	配置文件介绍	36
5.1	Openni.ini	36
5.2	Orbbec.ini	36
6	配置文件 API 接口.....	39
6.1	帧同步	39
6.2	镜像.....	39
6.3	获取设备的 Pid	40
6.4	设置分辨率	40
6.5	设置 Depth 最小值和最大值	42
6.6	设置 Depth 的传输格式	42
6.7	设置 Depth 软件滤波开关	43 42
6.8	设置 Depth 旋转开关	43
6.9	设置 HoleFilter	43
6.10	设置 IR 输出格式(OutputFormat)	43
7	结束.....	44

1 简述

该文档主要介绍 Openni2 SDK，为了使用户正确的、高效的在自己的项目中快速接入 OpenNI2 SDK，防止在使用 SDK 相关 API 接口的过程中由于不规范的调用而引起其他问题，故拟定次文档来规范 API 的调用。

2 openni2.0 架构

OpenNI2 本身是按照分层设计的，OpenNI 框架如下。

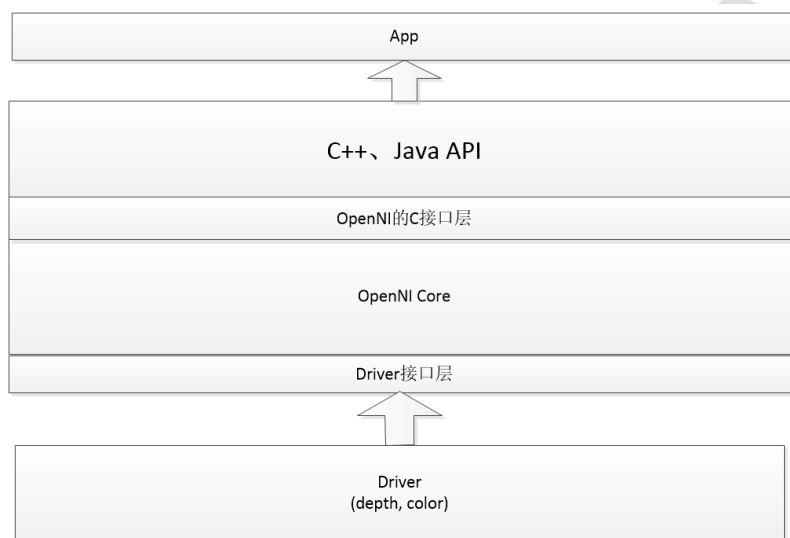


图 1: openni2.0 架构

Openni2.0 Driver 层:

- 平台适配，主要是 XnLib 库，隔离 windows 和 linux 系统之间的差异，包括线程，内存管理，事件/互斥锁，定时器等。
- 适配 windows 和 linux 下 usb 通信接口，linux 使用 libusb 通信，windows 使用 orbbec 自己的驱动 obdrv.sys，对上层提供统一的接口。
- Usb 通信层，包括设备枚举，usb 控制命令实现，视频流的接收，解析。

Openni 层： 主要封装对外的获取视频流接口及参数设置的接口。

3 Openni2 对外的接口

3.1 视频数据接口

OpenNI2 SDK 获取图像数据获取方式有 2 种：主动获取(Fetch) 和回调函数(Callback)。图 2 和图 3 都是标准的 openni 获取摄像头数据的接口流程，可以根据需求选择。

3.1.1 主动获取

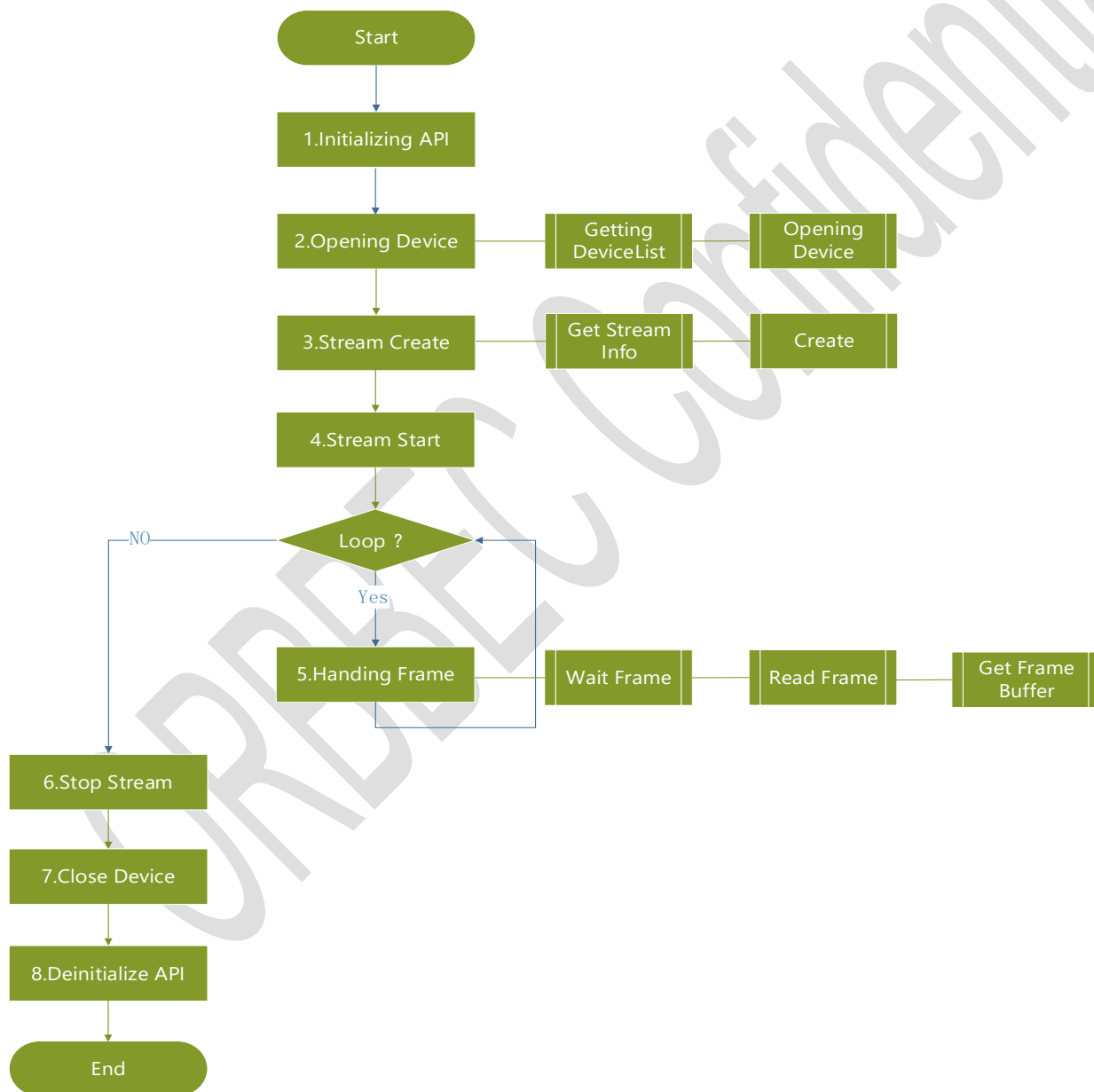


图 2：图像数据主动获取流程图

1. SDK Initializing

调用API	<code>OpenNI::initialize()</code>
详细描述	初始化OpenNI2 SDK，并枚举Orbbec的usb设备。
具体实现	<pre>Status rc = OpenNI::initialize(); if (rc != STATUS_OK) { printf("Initialize failed\n%s\n", OpenNI::getExtendedError()); return 1; }</pre>

2. Opening Device

调用API	<code>device.open(ANY_DEVICE);</code>
详细描述	默认打开枚举到的第一设备，如果存在多个设备，参数传输枚举到的URI。
具体实现	<pre>Device device; rc = device.open(ANY_DEVICE); if (rc != STATUS_OK) { printf("Couldn't open device\n%s\n", OpenNI::getExtendedError()); return 2; }</pre>

3. Stream Create

调用API	<code>videoStream.create(device, SENSOR_DEPTH);</code>
详细描述	创建数据流VideoStream,参数Device、SensorType。SensorType表示将要创建的数据流类型。 SensorType: SENSOR_IR(IR数据流),SENSOR_COLOR(RGB数据流,该数据流的输出仅Astra系列设备支持),SENSOR_DEPTH(Depth数据流)。
具体实现	<pre>if (device.getSensorInfo(SENSOR_DEPTH) != NULL) { rc = depth.create(device, SENSOR_DEPTH); if (rc != STATUS_OK) { printf("Couldn't create depth stream\n%s\n", OpenNI::getExtendedError()); return 3; } }</pre>

4. Stream Start

调用API	<code>videoStream.start();</code>
-------	-----------------------------------



详细描述	开始输数流数据。
具体实现	<pre>rc = depth.start(); if (rc != STATUS_OK) { printf("Couldn't start the depth stream\n%s\n", OpenNI::getExtendedError()); return 4; }</pre>

5. Handing Frame

调用API	<pre>1.OpenNI::waitForAnyStream(&pStream, 1, &changedStreamDummy, SAMPLE_READ_WAIT_TIMEOUT); 2.depth.readFrame(&frame);</pre>
详细描述	主动获取流数据，并等待SAMPLE_READ_WAIT_TIMEOUT超时时间(一般2000ms),如果获取到数据流，读取数据帧。 注意：该过程是耗时操作，需要开启子线程去执行。
具体实现	<pre>int changedStreamDummy; VideoStream* pStream = &depth; rc = OpenNI::waitForAnyStream(&pStream, 1, &changedStreamDummy, SAMPLE_READ_WAIT_TIMEOUT); if (rc != STATUS_OK) { printf("Wait failed! (timeout is %d ms)\n%s\n", SAMPLE_READ_WAIT_TIMEOUT, OpenNI::getExtendedError()); continue; } rc = depth.readFrame(&frame); if (rc != STATUS_OK) { printf("Read failed!\n%s\n", OpenNI::getExtendedError()); continue; } PixelFormat pFormat= frame.getVideoMode().getPixelFormat(); if (pFormat != PIXEL_FORMAT_DEPTH_1_MM && pFormat != PIXEL_FORMAT_DEPTH_100_UM) { printf("Unexpected frame format\n"); continue; } //获取到Depth数据 DepthPixel* pDepth = (DepthPixel*)frame.getData();</pre>

6. Stop Stream

调用API	videoStream.stop();
详细描述	停止数据流, 停止数据流后可以再start stream。

7. Destroy Stream

调用API	videoStream.destroy();
详细描述	销毁数据流, 如果需要重新获取数据流需要从3创建VideoStream开始执行。

8. 7.Close Device

调用API	Device.close();
详细描述	关闭以打开的Device, 重新获取数流从2步骤开始执行。

9. Deinitialize API

调用API	OpenNI::shutdown();
详细描述	释放OpenNI2 SDK, Openni 2 SDK反初始化, 释放SDK内存等资源, 再次操作Device需重新初始化SDK执行。

3.1.2 被动获取

被动获取既回调函数数据, 获取视频数据的流程图如下。和主动获取大部分是类似, 通过回调函数获取视频数据需要注册一个回调, 参考 demo: EventBasedRead。

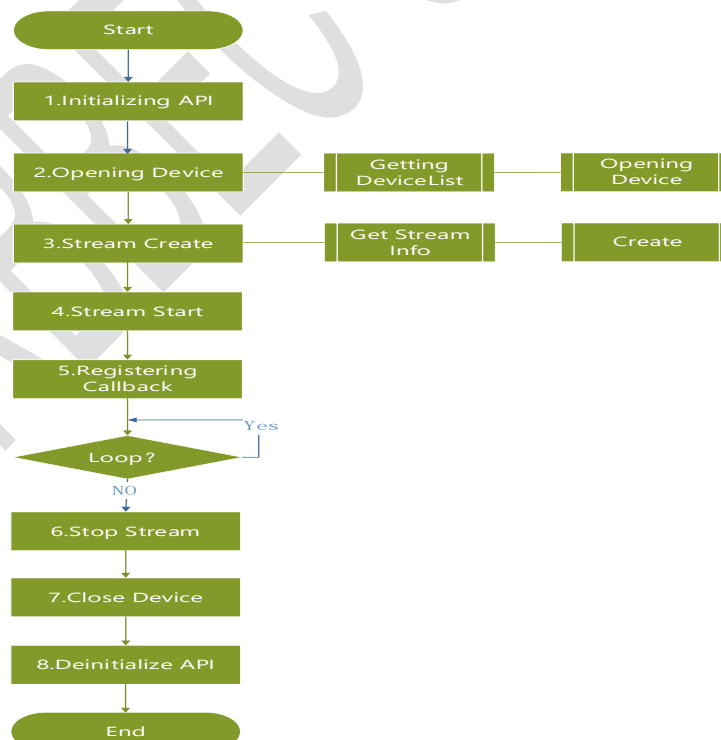


图 3：使用回调函数获取视频数据流程图

使用回调方式获取数据的例子，和主动获取数据大部分都一样，只有第 5 步不一样，在主动获取数据时，使用的是轮询的方法来获取数据，而被动获取数据，使用回调的方法实现。下面主要看下第 5 步骤具体实现。

1.实现数据流回调函数

调用函数	<code>public VideoStream::NewFrameListener</code>
具体实现	<pre> class PrintCallback : public VideoStream::NewFrameListener { public: void onNewFrame(VideoStream& stream) { stream.readFrame(&m_frame); analyzeFrame(m_frame); //打印收到的帧数据 } private: VideoFrameRef m_frame; }; </pre>

2.设置回调函数

调用函数	<code>videoStream.addNewFrameListener(NewFrameListener* pListener)</code>
具体实现	<pre> PrintCallback depthPrinter; // Register to new frame videoStream.addNewFrameListener(&depthPrinter); </pre>

3. 具体 EventBasedRead 源码参考文件



3.1.3 获取 IR

IR 图分为散斑 IR 和纯净 IR，散斑 IR 是指的 IR 图像上有激光散斑，纯净 IR 就是普通的 IR 图，一般识别算法都需要纯净的 IR 图。

3.1.3.1 散斑 IR

获取 IR 的方法和获取 Depth 的方法一样，分为主动获取和被动获取，和获取 depth 数据比较，只需要将 create 流的类型修改成 IR。实现如下：

调用API	<code>videoStream.create(device, SENSOR_IR);</code>
详细描述	创建数据流VideoStream,参数Device、SensorType。SensorType表示将要创建的数据流类型。 SensorType: SENSOR_IR(IR数据流),SENSOR_COLOR(RGB数据流,该数据流的输出仅Astra系列设备支持),SENSOR_DEPTH(Depth数据流)。
具体实现	<pre> if (ir.getSensorInfo(SENSOR_IR) != NULL) { </pre>



	<pre>rc = ir.create(device, SENSOR_IR); if (rc != STATUS_OK) { printf("Couldn't create ir stream\n%s\n",OpenNI::getExtendedError()); return 3; } }</pre>
--	--

3.1.3.2 右散斑 IR

获取右散斑 IR 的方法和获取 IR 的方法一样，分为主动获取和被动获取，和获取 IR 数据比较，只需要将 create 流的类型修改成右 IR,目前只有个别项目支持左右 IR 同时输出，如双目扫地机器人。
实现如下：

调用API	videoStream.create(device, SENSOR_RIGHT_IR);
详细描述	创建数据流VideoStream,参数Device、SensorType。SensorType表示将要创建的数据流类型。 SensorType: SENSOR_IR(IR数据流),SENSOR_COLOR(RGB数据流,该数据流的输出仅Astra系列设备支持),SENSOR_DEPTH(Depth数据流), SENSOR_RIGHT_IR(右IR数据流)
具体实现	<pre>if (ir.getSensorInfo(SENSOR_IR) != NULL) { rc = ir.create(device, SENSOR_RIGHT_IR); if (rc != STATUS_OK) { printf("Couldn't create ir stream\n%s\n",OpenNI::getExtendedError()); return 3; } }</pre>

3.1.3.3 纯净 IR

纯净 IR 图不是所有的设备都支持输出，如果要使用纯净 IR，需要了解产品是否支持。纯净 IR 速出，首先需要关闭 LDP，关闭激光，最后打开泛光灯即可输出纯净 IR 数据。如 Gemini、S1 等带有泛光灯的设备支持。如果同时在开深度和 IR 的情况下，切换 depth 和纯净 IR,需要在这两路流获取到后，然后切换。

获取纯净 IR 的实现流程：



3.1.3.4 纯净 IR 切换散斑 IR

纯净 IR 切换散斑 IR，**注意切换过程不需要关闭数据流**。首先需要关闭泛光灯，打开 LDP，最后打开激光即可输出散斑 IR 数据。如 Gemini、S1 等带有泛光灯的设备支持。纯净 IR 切换散斑 IR 的实现流程：



3.1.4 获取 Color (openni)

Orbbec 设备 color rgb 分为两类，一类设备 color 是使用 openni 协议传输的，如 astra，astra Mini 等，一类设备 color 是使用 uvc 协议传输的，如：astra pro，astra pro S1，deeyea，canglong，大白，gemini 等设备。

使用 UVC 协议的 color rgb，建议在 windows 上使用 windows media foundation(WMF)获取 UVC 数据，在 linux 或 Arm 上使用 V4L2 获取 UVC 数据。

使用 openni 获取 color 的方法和获取 Depth 的方法一样，分为主动获取和被动获取，和获取 depth 数据比较，只需要将 create 流的类型修改成 color。 实现如下：

调用API	<code>videoStream.create(device, SENSOR_COLOR);</code>
详细描述	创建数据流VideoStream,参数Device、SensorType。SensorType表示将要创建的数据流类型。 SensorType: SENSOR_IR(IR数据流),SENSOR_COLOR(RGB数据流,该数据流的输出仅Astra系列设备支持),SENSOR_DEPTH(Depth数据流)。
具体实现	<pre> if (color.getSensorInfo(SENSOR_COLOR) != NULL) { rc = ir.create(device, SENSOR_COLOR); if (rc != STATUS_OK) { printf("Couldn't create COLOR stream\n%s\n",OpenNI::getExtendedError()); return 3; } } </pre>

3.2 Openni 常用接口介绍

3.2.1 Device 类

3.2.1.1 open 函数

调用API	<code>Status openni::Device::open(const char * uri);</code>
详细描述	函数功能：open 设备； 参 数：uri：枚举到的设备的字符串，或者 ANY_DEVICE，参数为 ANY_DEVICE； 返回值：表示open是否成功，STATUS_OK：表示成功，其它表示失败。

3.2.1.2 close() 函数

调用API	<code>openni::Device::close ();</code>
详细描述	函数功能：关闭设备；

参 数：无；
返回 STATUS_OK：表示成功，其它表示失败。

3.2.1.3 setDepthColorSyncEnabled 函数

调用API	Status openni::Device::setDepthColorSyncEnabled (bool isEnabled);
详细描述	函数功能：设置帧同步，color 使用 openni 的设备才能帧同步，color 使用 uvc 协议不支持帧同步； 参数：true: 表示开启帧同步，false: 表示关闭帧同步； 返回 STATUS_OK：表示成功，其它表示失败。

3.2.1.4 setImageRegistrationMode 函数

调用API	Status openni::Device::setImageRegistrationMode (ImageRegistrationMode mode);
详细描述	函数功能：开启硬件 D2C 或关闭硬件 D2C 函数参数：mode: <pre>typedef enum { IMAGE_REGISTRATION_OFF = 0, //关闭硬件 D2C IMAGE_REGISTRATION_DEPTH_TO_COLOR = 1, //开启硬件 D2C } ImageRegistrationMode;</pre> 返回值： STATUS_OK：表示成功，其它表示失败。

3.2.1.5 setProperty 函数[1]

调用API	Status openni::Device::setProperty (int propertyId, const void* data, int dataSize);
详细描述	函数功能：设置参数 参数： [in] propertyId: 命令 id; [in] data: 设置参数的 Buff; [in] dataSize : 参数 Size 返回值：返回参数设置的状态，STATUS_OK：表示成功，其它表示失败。

3.2.1.6 setProperty 函数[2]

调用API	Status openni::Device::setProperty (int propertyId, const T & value);
详细描述	函数功能：设置参数 参数： [in] propertyId: 命令 id;



	[in] value: 表示设置的值, 适合于传入一个结构体 对象的引用。 返回值: 返回参数设置的状态, STATUS_OK: 表示成功, 其它表示失败。
--	---

3.2.1.7 getProperty 函数[1]

调用API	template<class T > Status openni::Device:: getProperty (int propertyId, void* data, int* dataSize) const;
详细描述	函数功能: 获取参数 参数: [in] propertyId: 命令 id; [out] data: 保存获取参数的 buff; [in,out] dataSize: 获取参数的 Size 返回值: STATUS_OK: 表示成功, 其它表示失败。

3.2.1.8 getProperty 函数[2]

调用API	template<class T > Status openni::Device::getProperty (int propertyId , T* value) const;
详细描述	函数功能: 获取参数 参数: [in] propertyId: 命令 id; [in, out] value :获取参数的指针, 这个适合参数是一个结构体的情况。 返回值: STATUS_OK: 表示成功, 其它表示失败。

3.2.1.9 getDeviceInfo 函数

调用API	const DeviceInfo&openni::Device::getDeviceInfo () const
详细描述	函数功能: 获取设备信息 返回值: 返回 openni::DeviceInfo 的对象, 这个类的定义参考 3.2.4 DeviceInfo 类。

3.2.2 DeviceConnectedListener 类

调用API	virtual void openni :: OpenNI :: DeviceConnectedListener ::onDeviceConnected(const DeviceInfo*);
详细描述	函数功能: 接收设备的连接事件, 具体使用方法参考 3.1.2 被动获取的 Demo 代码。

3.2.3 DeviceDisconnectedListener 类

调用API	virtual void openni::OpenNI::DeviceDisconnectedListener::onDeviceDisconnected(const DeviceInfo*);
详细描述	函数功能: 接收设备的断开事件, 具体使用方法参考 3.1.2 被动获取的 Demo 代码。

3.2.4 DeviceInfo 类

详细描述	通过 Device 类的 getDeviceInfo 函数可以获取 DeviceInfo 的对象，然后就可以设备的一些信息。 比如： <pre>const char * getName () const //获取设备名 const char * getUri () const //获取设备的 URI uint16_t getUsbProductId () const //获取设备 Pid uint16_t getUsbVendorId () const //获取设备的 Vid</pre> 调用方法举例： <pre>//获取设备的 PID，g_device 打开的设备 g_device.getDeviceInfo().getUSBProductId();</pre>
------	--

3.2.5 DeviceStateChangedListener 类

调用API	virtual void openni::OpenNI:: DeviceStateChangedListener::onDeviceStateChanged (const DeviceInfo*, DeviceState);
详细描述	这个类用来接收设备状态的改变事件。 函数功能：接收设备状态的改变事件，具体使用方法参考 3.1.2 被动获取的 Demo 代码。

3.2.6 NewFrameListener 类

调用API	virtual void openni::VideoStream::NewFrameListener::onNewFrame(VideoStream&)
详细描述	函数功能： 应用程序注册一个帧回调函数后，数据通过这个函数回调到应用程序。 接收数据的回调函数，用在被动获取数据时，具体使用方法参考 3.1.2 被动获取的 Demo 代码。

3.2.7 OpenNI 类

3.2.7.1 initialize 函数

调用API	static Status openni::OpenNI::initialize ()
详细描述	函数功能：初始化 openni SDK 库 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.7.2 shutdown 函数

调用API	static Status openni::OpenNI::shutdown ()
详细描述	函数功能：停止使用 openni SDK，停止流和关闭设备，释放资源。 注意：如果在调用其它 API 接口时，return 直接返回退出程序，必须要调用该函数，防止资源未释放。

3.2.7.3 waitForAnyStream 函数

调用API	static Status openni::OpenNI::waitForAnyStream(VideStream** pStream,int streamCount, int* pReadyStreamIndex, int timeout = TIMEOUT_FOREVER)
详细描述	函数功能：主动获取数据时，等待是否有数据，当没有流数据时，该函数会阻塞。具体用法参考 3.1.1 主动获取的 Demo。 函数参数： [in] pStreams: 保存监听的视频流的数组，可以同时监听多个流。 [in] streamCount: 监听流的数量 [out] pReadyStreamIndex: 返回第一个有帧数据的视频流索引 [in] timeout: 超时时间，单位毫秒，默认值：TIMEOUT_FOREVER

3.2.7.4 getVersion 函数

调用API	static Version openni::OpenNI::getVersion()
详细描述	函数功能：获取 Openni 的版本 返回值：返回 openni 的版本信息。

3.2.7.5 enumerateDevices 函数

调用API	static void openni::OpenNI::enumerateDevices(Array<DeviceInfo>* deviceInfoList)
详细描述	函数功能：枚举 orbdec 的所有设备列表； 参数：[in, out] deviceInfoList: 枚举到的设备数组。

3.2.7.6 addDeviceConnectedListener 函数

调用API	static Status openni::OpenNI::addDeviceConnectedListener(DeviceConnectedListener* pListener)
详细描述	函数功能：注册设备连接事件；使用例子参考：3.1.2 被动获取中的 Demo。 参数： pListener: 加入到监听列表中设备连接类对象指针，参考：3.2.2 DeviceConnectedListener 类



	返回值：STATUS_OK：表示成功，其它表示失败。
--	----------------------------

3.2.7.7 removeDeviceConnectedListener 函数

调用API	static void openni::OpenNI::removeDeviceConnectedListener(DeviceConnectedListener* pListener)
详细描述	函数功能：从设备连接列表中删除对设备连接的监听，是 addDeviceConnectedListener 的相反功能函数。 参数：pListener：监听列表中设备连接类对象指针，参考：3.2.2 DeviceConnectedListener 类。 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.7.8 addDeviceDisconnectedListene 函数

调用API	static status openni::OpenNI::addDeviceDisconnectedListener(DeviceDisconnectedListener * pListener)
详细描述	函数功能：注册设备断开事件；使用例子参考：3.1.2 被动获取中的 Demo。 参数：pListener：加入到监听列表中设备断开类对象指针。 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.7.9 removeDeviceDisconnectedListener 函数

调用API	static voidopenni::OpenNI::removeDeviceDisconnectedListener(DeviceDisconnectedListener * pListener)
详细描述	函数功能：从设备连接列表中删除对设备断开的监听。 参数：pListener：监听列表中设备断开类对象指针。 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.7.10 addDeviceStateChangedListener 函数

调用API	static status openni::OpenNI::addDeviceStateChangedListener(DeviceStateChangedListener * pListener)
详细描述	函数功能：注册设备状态改变事件，3.1.2 被动获取中的 Demo。 参数：pListener：设备状态改变类对象指针。 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.7.10 removeDeviceStateChangedListener 函数

调用API	static void openni::OpenNI::removeDeviceStateChangedListener(DeviceStateChangedListener * pListener)
详细描述	函数功能：从设备连接列表中删除对设备状态改变的监听。 参数：pListener：设备状态改变类对象指针。 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.8 VideoFrameRef 类

3.2.8.1 getData 函数

调用API	const void* openni::VideoFrameRef::getData () const
详细描述	函数功能：获取视频帧数据的 Buff

3.2.8.2 getDataSize 函数

调用API	int openni::VideoFrameRef::getDataSize () const
详细描述	获取视频帧数据的长度。

3.2.8.2 getHeight 函数

调用API	int openni::VideoFrameRef::getHeight () const
详细描述	函数功能：获取视频帧图像的高。

3.2.8.3 getWidth 函数

调用API	int openni::VideoFrameRef::getWidth () const
详细描述	函数功能：获取视频帧图像的宽。

3.2.8.4 getTimestamp 函数

调用API	uint64_t openni::VideoFrameRef::getTimestamp () const
-------	--



详细描述	函数功能：获取视频帧的时间戳，单位微秒。
------	----------------------

3.2.8.5 getVideoMode 函数

调用API	const VideoMode&openni::VideoFrameRef::getVideoMode () const
详细描述	函数功能：返回设备的 VideoMode，通过 VideoMode，可以获取设备当前的分辨率等信息，具体可以参考 3.2.9 VideoMode 类。

3.2.9 VideoMode 类

3.2.9.1 setPixelFormat 函数

调用API	void openni::VideoMode::setPixelFormat (PixelFormat format)
详细描述	函数功能：设置视频流输出格式，具体方法，参考 6.4 设置分辨率 参数： [in] format：视频流输出格式 <pre>typedef enum { // Depth ONI_PIXEL_FORMAT_DEPTH_1_MM = 100, //Depth 1mm单位输出 ONI_PIXEL_FORMAT_DEPTH_100_UM = 101, //Depth 100um 单位输出 ONI_PIXEL_FORMAT_SHIFT_9_2 = 102, //输出视差 //ONI_PIXEL_FORMAT_SHIFT_9_3 = 103, // Color ONI_PIXEL_FORMAT_RGB888 = 200, //openni 的color 输出RGB888 ONI_PIXEL_FORMAT_YUV422 = 201, //color 输出YUV422(UYVY) //ONI_PIXEL_FORMAT_GRAY8 = 202, //ONI_PIXEL_FORMAT_GRAY16 = 203, //ONI_PIXEL_FORMAT_JPEG = 204, //ONI_PIXEL_FORMAT_YUYV = 205, } OniPixelFormat;</pre>

3.2.9.2 setResolution 函数

调用API	void openni::VideoMode::setResolution (int resolutionX, int resolutionY)
详细描述	函数功能：设置分辨率，具体方法，参考 6.4 设置分辨率。

3.2.10 VideoStream 类

3.2.10.1 addNewFrameListener 函数

调用API	Status openni::VideoStream::addNewFrameListener (NewFrameListener * pListener)
详细描述	函数功能：被动获取数据设置回调函数， 参考 3.1.2 被动获取数据的 Demo 参数： [in] pListener：指向 NewFrameListener 对象的指针，具体参考：3.2.6 NewFrameListener 类 返回值：返回值：STATUS_OK：表示成功，其它表示失败。

3.2.10.2 removeNewFrameListener 函数

调用API	void openni::VideoStream::removeNewFrameListener (NewFrameListener * pListener)
详细描述	函数功能：被动获取数据 Remove 回调函数， 参考 3.1.2 被动获取数据的 Demo 参数： [in] pListener：指向 NewFrameListener 对象的指针，具体参考：3.2.6 NewFrameListener 类 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.10.3 create 函数

调用API	Status openni::VideoStream::create (const Device & device, SensorType sensorType)
详细描述	函数功能：create 视频流 参数： [in] device： create 流的 Device 引用；参考 3.1 视频数据接口； [in] sensorType：视频流的类型，取值如下： <pre>typedef enum { SENSOR_IR = 1, SENSOR_COLOR = 2, SENSOR_DEPTH = 3, } SensorType;</pre> 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.10.4 start 函数

调用API	Status openni::VideoStream::start ()
详细描述	函数功能：开始启动视频流 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.10.5 stop 函数

调用API	void openni::VideoStream::stop ()
详细描述	函数功能：停止视频流 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.10.6 setVideoMode 函数

调用API	Status openni::VideoStream::setVideoMode (const VideoMode & videoMode)
详细描述	函数功能：设置视频模式，可以设置图像输出格式，分辨率等，可以参考 VideoMode 类及 6.4 设置分辨率。 参数：[in] videoMode: 视频模式； 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.10.7 getVideoMode 函数

调用API	VideoMode openni::VideoStream::getVideoMode () const
详细描述	函数功能：获取视频模式，包括视频流输出格式，分辨率和帧率； 返回值：返回获取到 VideoMode。

3.2.10.8 readFrame 函数

调用API	Status openni::VideoStream::readFrame (VideoFrameRef * pFrame)
详细描述	函数功能：获取一帧数据； 参数:[out] pFrame: 保存帧 Buff 的指针，参考：3.2.8 VideoFrameRef 类； 返回值：STATUS_OK：表示成功，其它表示失败。

3.2.10.9 setProperty 函数[1]

调用API	<code>status openni::VideoStream::setProperty (int propertyId, const void * data, int dataSize)</code>
详细描述	函数功能：通过 VideoStream 类设置设备的流属性参数。 参数： [in] propertyId: 命令 id; [in] data: 设置参数的 Buff [in] dataSize : 参数 Size 返回值：返回参数设置的状态，STATUS_OK: 表示成功，其它表示失败。

3.2.10.10 setProperty 函数[2]

调用API	<code>template<class T> Status openni::VideoStream::setProperty (int propertyId, const T & value)</code>
详细描述	函数功能：通过 VideoStream 类设置设备的流属性参数。 参数： [in] propertyId: 命令 id; [in] value: 表示设置的值，适合于传入一个结构体 对象的引用。 返回值：返回参数设置的状态，STATUS_OK: 表示成功，其它表示失败。

3.2.10.11 getProperty 函数[1]

调用API	<code>Status openni::VideoStream::getProperty (int propertyId, void * data, int * dataSize) const</code>
详细描述	函数功能：通过 VideoStream 类获取设备的流属性参数。 参数： [in] propertyId: 命令 id; [out] data: 保存获取参数的 buff; [in,out] dataSize: 获取参数的 Size 返回值：STATUS_OK: 表示成功，其它表示失败。

3.2.10.12 getProperty 函数[2]

调用API	<code>template<class T> Status openni::VideoStream::getProperty(int propertyId, T * value) const</code>
详细描述	函数功能：通过 VideoStream 类获取设备的流属性参数。 参数： [in] propertyId: 命令 id; [in, out] value :获取参数的指针，这个适合参数是一个结构体的情况。 返回值：STATUS_OK: 表示成功，其它表示失败。

3.2.11 Recorder 类

Recorder 类的函数用来录制 Oni 文件。

3.2.11.1 attach 函数

调用API	Status openni::Recorder::attach(VideoStream & stream, bool allowLossyCompression = false)
详细描述	函数功能：Recorder 和 stream 关联。 参数： [in] stream: 关联的视频流; [in] allowLossyCompression: 是否压缩, true: 表示压缩, false: 表示不压缩。 返回值：STATUS_OK: 表示成功, 其它表示失败。

3.2.11.2 create 函数

调用API	Status openni::Recorder::create (const char * fileName)
详细描述	函数功能：初始化 Recorder 参数：[in] fileName: 保存录像的文件名; 返回值：STATUS_OK: 表示成功, 其它表示失败。

3.2.11.3 start 函数

调用API	Status openni::Recorder::start ()
详细描述	函数功能：启动录像。

3.2.11.4 stop 函数

调用API	void openni::Recorder::stop ()
详细描述	函数功能：停止录像。

3.2.11.5 destroy 函数

调用API	void openni::Recorder::destroy ()
详细描述	函数功能：destroy Recorder 对象。

3.2.12 CameraSettings 类

有些设备如：astra,astra mini, astra mini S, color 使用的是 openni 的协议传输，调节 color 的 AE 的接口如下：
设备端默认使用的是自动 AE。

3.2.12.1 getAutoExposureEnabled 函数

调用API	bool getAutoExposureEnabled()
详细描述	函数功能：获取当前 AE 的状态； 返回值：true:表示自动 AE，false: 表示手动 AE

3.2.12.2 setAutoExposureEnabled 函数

调用API	Status setAutoExposureEnabled(bool enabled)
详细描述	函数功能：设置 AE 的状态； 参数：true:表示自动 AE，false: 表示手动 AE 返回值：成功：STATUS_OK，其它值：表示设置失败；

3.2.12.3 setGain 函数

调用API	Status openni::CameraSettings::setGain (int gain)
详细描述	函数功能：设置增益值； 参数：gain: 表示增益； 返回值：成功：STATUS_OK，其它值：表示设置失败；

3.2.12.4 setExposure 函数

调用API	Status openni::CameraSettings::setExposure (int exposure)
详细描述	函数功能：设置曝光值； 参数：exposure 表示曝光； 返回值：成功：STATUS_OK，其它值：表示设置失败；

3.2.12.5 getGain 函数

调用API	int openni::CameraSettings::getGain ()
详细描述	函数功能：获取当前 Gain 值； 返回值：获取的 Gain 值

3.2.12.5 getExposure 函数

调用API	int openni::CameraSettings::getExposure ()
详细描述	函数功能：获取当前 exposure 值； 返回值：获取的 exposure 值。

Astra, Astra mini, Astra miniS, lunaP1 这些使用 openni 传输 color 的设备，手动调节 AE 的方法如下：
首先将 AE 切换到手动模式，调用例子如下：

`g_colorStream.getCameraSettings()->setAutoExposureEnabled(false);` // g_colorStream: 为通过 openni 创建的 color 流。

然后在设置曝光和增益值：

`int exposure = xx; //设置的曝光值`

`g_colorStream.getCameraSettings()->setExposure(exposure);`

`int gain = xx; //设置的增益值`

`g_colorStream.getCameraSettings()->setGain(gain);`

3.2.11.6 录像例子

源码：

```
#include <stdio.h>
#include <OpenNI.h>
#include <vector>
#include <memory>
#include "OniSampleUtilities.h"

#define SAMPLE_READ_WAIT_TIMEOUT 2000 //2000ms
using namespace openni;

int main()
{
    Status rc = OpenNI::initialize();
    if (rc != STATUS_OK)
    {
        printf("Initialize failed\n%s\n", OpenNI::getExtendedError());
        return 1;
    }
}
```

```
Device device;
rc = device.open(ANY_DEVICE);
if (rc != STATUS_OK)
{
    printf("Couldn't open device\n%s\n", OpenNI::getExtendedError());
    return 2;
}

VideoStream depth;
if (device.getSensorInfo(SENSOR_DEPTH) != NULL)
{
    rc = depth.create(device, SENSOR_DEPTH);
    if (rc != STATUS_OK)
    {
        printf("Couldn't create depth stream\n%s\n", OpenNI::getExtendedError());
        return 3;
    }
}

Recorder recorder;
recorder.create("111.oni");
recorder.attach(depth, false);
recorder.start();

rc = depth.start();
if (rc != STATUS_OK)
{
    printf("Couldn't start the depth stream\n%s\n", OpenNI::getExtendedError());
    return 4;
}

VideoFrameRef frame;
while (!wasKeyboardHit())
{
    int changedStreamDummy;
    VideoStream* pStream = &depth;
    rc = OpenNI::waitForAnyStream(&pStream, 1, &changedStreamDummy, SAMPLE_READ_WAIT_TIMEOUT);
    if (rc != STATUS_OK)
    {
        printf("Wait failed! (timeout is %d ms)\n%s\n", SAMPLE_READ_WAIT_TIMEOUT, OpenNI::getExtendedError());
        continue;
    }

    rc = depth.readFrame(&frame);
    if (rc != STATUS_OK)
    {
        printf("Read failed!\n%s\n", OpenNI::getExtendedError());
        continue;
    }

    if (frame.getVideoMode().getPixelFormat() != PIXEL_FORMAT_DEPTH_1_MM && frame.getVideoMode().getPixelFormat() !=
    PIXEL_FORMAT_DEPTH_100_UM)
    {
        printf("Unexpected frame format\n");
        continue;
    }

    DepthPixel* pDepth = (DepthPixel*)frame.getData();
    int middleIndex = (frame.getHeight()+1)*frame.getWidth()/2;
    printf("[%08llu] %8d\n", (long long)frame.getTimestamp(), pDepth[middleIndex]);
}

depth.stop();
recorder.stop();
recorder.destroy();
depth.destroy();
device.close();
OpenNI::shutdown();
```

```
return 0;
}
```

3.3 扩展 Api 接口

扩展 API 是 OpenNI 协议的扩展，是 Orbbec 的私有接口协议，用来控制 Orbbec 的 3D 摄像头，在调用相关扩展 API 之前，需要先确保 OpenNI2.0-SDK Initialize 成功，设备 Open 成功，下面详细介绍所有扩展 API 的使用方法。

3.3.1 获取固件版本号

详细描述	获取固件版本号
具体实现	<pre>char strPlatformString[128] = {0}; int size = sizeof(strPlatformString); openni::Status rc = Device.getProperty(XN_MODULE_PROPERTY_SENSOR_PLATFORM_STRING, strPlatformString, &size); if (strPlatformString[0] != '\0') { printf("%s", strPlatformString); }</pre>

3.3.2 获取设备序列号

详细描述	获取设备序列号
具体实现	<pre>char serNumber[12]={0}; int dataSize=sizeof(serNumber); openni::Status rc = Device.getProperty(OBEXTENSION_ID_SERIALNUMBER, (uint8_t *)&serNumber,&dataSize);</pre>

3.3.3 获取设备类型

详细描述	获取设备类型
具体实现	<pre>char deviceType[32]={0}; int dataSize=sizeof(deviceType); openni::Status rc = Device.getProperty(OBEXTENSION_ID_DEVICE_TYPE, (uint8_t *)&deviceType, &dataSize);</pre>

3.3.4 获取相机保存标定参数

详细描述	Astra 系列只有 Astra S 在出厂时有保存标定参数。其他型号可以手动标定并按第 4 节做法保存到设备。如果设备没有标定，先手动标定，再获取标定参数。其它二代设备，如 Deeyea，苍龙 1 号，大白，Gemini，Projector 等出厂时已经烧录标定参数。
具体实现	1. 标定参数结构体： <pre>typedef struct OBCameraParams { float l_intr_p[4]; // [fx, fy, cx, cy] // Depth 相机的内参 float r_intr_p[4]; // [fx, fy, cx, cy] // color 相机的内参 float r2l_r[9]; // [r00, r01, r02; r10, r11, r12; r20, r21, r22] // 旋转矩阵 float r2l_t[3]; // [t1, t2, t3] // 平移矩阵 float l_k[5]; // [k1, k2, p1, p2, k3] // Depth 的畸变参数 float r_k[5]; // color 的畸变参数 // int is_mirror; } OBCameraParams;</pre> 2. 具体实现接口： <pre>OBCameraParams cameraParam; int dataSize = sizeof(cameraParam); memset(&cameraParam, 0, sizeof(cameraParam)); openni::Status rc = Device.getProperty(openni::OBEXTENSION_ID_CAM_PARAMS, (uint8_t*) &cameraParam, &dataSize);</pre>

3.3.5 获取哈雷相机保存标定参数

1. 接口说明：

详细描述	该接口目前只能用于哈雷模组（Orbbec Dabai DW/ Orbbec Dabai DCW），不能使用“3.3.4”接口读取相机标定参数。 因哈雷模组存在多分辨率的相机参数，所用需要通过深度 <code>XnCameraParamsDepthRes</code> 和 <code>Rgb XnCameraParamsColorRes</code> 的不同参数分辨率匹配返回正确的相机参数。
具体实现	1. 参数结构体： <pre>// 深度参数分辨率 typedef enum { XN_CAMERA_PARAMS_DEPTH_RES_1024_768 = 0, XN_CAMERA_PARAMS_DEPTH_RES_512_384 = 1, XN_CAMERA_PARAMS_DEPTH_RES_640_480 = 2, XN_CAMERA_PARAMS_DEPTH_RES_320_240 = 3, XN_CAMERA_PARAMS_DEPTH_RES_160_120 = 4, XN_CAMERA_PARAMS_DEPTH_RES_480_360 = 5, XN_CAMERA_PARAMS_DEPTH_RES_240_180 = 6, XN_CAMERA_PARAMS_DEPTH_RES_640_360 = 7,</pre>

```

XN_CAMERA_PARAMS_DEPTH_RES_320_180 = 8,
} XnCameraParamsDepthRes;

//Rgb参数分辨率
typedef enum {
    XN_CAMERA_PARAMS_COLOR_RES_DEFAULT = 0,
    XN_CAMERA_PARAMS_COLOR_RES_640_360 = 1,
    XN_CAMERA_PARAMS_COLOR_RES_320_180 = 2,
} XnCameraParamsColorRes;

typedef struct OBCameraParams
{
    float l_intr_p[4];//[fx,fy,cx,cy] //Depth 相机的内参
    float r_intr_p[4];//[fx,fy,cx,cy] //color 相机的内参
    float r2l_r[9];//[r00,r01,r02;r10,r11,r12;r20,r21,r22] //旋转矩阵
    float r2l_t[3];//[t1,t2,t3] //平移矩阵
    float l_k[5];//[k1,k2,p1,p2,k3] //Depth 的畸变参数
    float r_k[5]; //color 的畸变参数
    //int is_mirror;
}OBCameraParams;

typedef struct
{
    XnCameraParamsDepthRes depthRes; //需要传参
    XnCameraParamsColorRes colorRes; //需要传参
    OBCameraParams params; //最终返回的相机参数
} OBCameraParamsData;

2. 示例代码（读取 DCW 的相机参数）：
OBCameraParamsData cameraParamData;
int dataSize = sizeof(OBCameraParamsData);
memset(&cameraParamData, 0, dataSize);
//设置正确匹配的参数分辨率参数
cameraParamData.depthRes = XN_CAMERA_PARAMS_DEPTH_RES_640_480;
cameraParamData.colorRes = XN_CAMERA_PARAMS_COLOR_RES_DEFAULT;
openni::Status rc = Device.getProperty(openni::OBEXTENSION_ID_CAM_PARAMS, (uint8_t
*)&cameraParamData, &dataSize);
if (rc != openni::STATUS_OK)
{
    printf("Error: %s\n", openni::OpenNI::getExtendedError());
}

```



2. DCW模组XnCameraParamsDepthRes与XnCameraParamsColorRes传参匹配说明:

序号	XnCameraParamsDepthRes	XnCameraParamsColorRes	备注
1	<code>XN_CAMERA_PARAMS_DEPTH_RES_1024_768 = 0,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_512_384 = 1,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_640_480 = 2,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_320_240 = 3,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_160_120 = 4,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_480_360 = 5,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_240_180 = 6</code>	<code>XN_CAMERA_PARAMS_COLOR_RES_DEFAULT = 0</code>	如果先择非匹配的传参, 读取相机参数接口返回异常。 <u>该参数的深度内参作为深度转点云的内参</u>
2	<code>XN_CAMERA_PARAMS_DEPTH_RES_640_360 = 7</code>	<code>XN_CAMERA_PARAMS_COLOR_RES_640_360 = 1</code>	如果先择非匹配的传参, 读取相机参数接口返回异常。
3	<code>XN_CAMERA_PARAMS_DEPTH_RES_320_180 = 8</code>	<code>XN_CAMERA_PARAMS_COLOR_RES_320_180 = 2</code>	<u>该参数的 color 内参作为 RGBD 数据的内参</u>

3. DW模组XnCameraParamsDepthRes与XnCameraParamsColorRes传参匹配说明:

序号	XnCameraParamsDepthRes	XnCameraParamsColorRes	备注
1	<code>XN_CAMERA_PARAMS_DEPTH_RES_1024_768 = 0,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_512_384 = 1,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_640_480 = 2,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_320_240 = 3,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_160_120 = 4,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_480_360 = 5,</code> <code>XN_CAMERA_PARAMS_DEPTH_RES_240_180 = 6</code>	<code>XN_CAMERA_PARAMS_COLOR_RES_DEFAULT = 0</code>	如果先择非匹配的传参, 读取相机参数接口返回异常。

3.3.6 获取 IR 增益

详细描述	获取 IR 增益
具体实现	<pre>int gain = 0; int dataSize = 4; openni::Status rc = Device.getProperty(OBEXTENSION_ID_IR_GAIN, (uint8_t*)&gain, &dataSize); printf("ir gain value:0x%x\n",gain);</pre>

3.3.7 设置 IR 增益

详细描述	设置 IR 增益
具体实现	<pre>int gain = 0; int dataSize = 4; openni::Status rc = Device.getProperty(OBEXTENSION_ID_IR_GAIN, (uint8_t*)&gain, &dataSize); printf("ir gain value:0x%x\n",gain); gain++; rc = Device.setProperty(OBEXTENSION_ID_IR_GAIN, (uint8_t*)&gain, dataSize);</pre>

3.3.8 获取 IR 曝光

详细描述	获取 IR 曝光
具体实现	<pre>int exposure = 0; int dataSize = 4; openni::Status rc = Device.getProperty(OBEXTENSION_ID_IR_EXP, (uint8_t*)&exposure, &dataSize); printf("ir exposure value:0x%x\n", exposure);</pre>

3.3.9 设置 IR 曝光

详细描述	设置 IR 曝光
具体实现	<pre>int exposure = 0; int dataSize = 4; openni::Status rc = Device.getProperty(OBEXTENSION_ID_IR_EXP, (uint8_t*)&exposure, &dataSize); printf("ir exposure value:0x%x\n", exposure); exposure++; rc = Device.setProperty(OBEXTENSION_ID_IR_EXP, (uint8_t*)&exposure, dataSize);</pre>

3.3.10 设置 LDP 开关 1

详细描述	注意: 1.该扩展 API 属于操作 Flash 接口,非必要需谨慎使用,尤其在使用过程中避免模组断电引发异常; 2.该扩展 API,只有一代 Astra 系列和一代升级的 Astra 系列某些带 LDP 产品有这个功能,如果不带 LDP 功能,不要调用; 3.如果要操作 LDP 的功能,优先使用“3.3.11” LDP 操作接口。
具体实现	<pre>int dataSize = 4;</pre>



	<pre>int ldp_en = enable; //0: 表示关闭 LDP; 1: 表示打开 LDP; openni::Status rc = Device.setProperty(OBEXTENSION_ID_LDP_EN, (uint8_t*)&ldp_en, dataSize);</pre>
--	---

3.3.11 设置 LDP 开关 2

详细描述	Astra pro plus, 大白, Gemini, projector, 等带 ldp 功能的设备支持这个接口, 这个接口断电不会保存, Astra 系列的设备不支持这个接口。
具体实现	<pre>int dataSize = 4; int ldp_enable = enable; //0: 表示关闭 LDP, 1: 表示打开 LDP openni::Status rc = Device.setProperty(XN_MODULE_PROPERTY_LDP_ENABLE, (uint8_t*)&ldp_enable, dataSize);</pre>

3.3.12 获取 LDP 使能状态

详细描述	S1, Astra pro plus, 大白, Gemini, projector, 等带 LDP 功能的设备, 支持这个接口, 这个接口断电不会保存, Astra 系列的设备不支持这个接口。
具体实现	<pre>int dataSize = 4; int ldp_enable = 0; openni::Status rc=Device.getProperty(XN_MODULE_PROPERTY_LDP_ENABLE, (uint8_t*)&ldp_enable, &dataSize);</pre>

3.3.13 设置 LDM 激光开关

详细描述	设置 LDM 激光开关, 如果该接口设备不支持, 请尝试调用“3.3.14”接口
具体实现	<pre>int dataSize = 4; int laser_en = enable; //1:表示开激光;0:表示关激光 openni::Status rc= Device.setProperty(OBEXTENSION_ID_LASER_EN, (uint8_t*)&laser_en, dataSize);</pre>

3.3.14 设置 LDM 激光开关

详细描述	LDM 激光开关有两个接口, 一个是通过扩展指令 OBEXTENSION_ID_LASER_EN 操作, 另外一个是通过 XN_MODULE_PROPERTY_EMITTER_STATE 操作。
具体实现	<pre>int dataSize = 4; int laser_en = enable; //1:表示开激光;0:表示关激光 openni::Status rc = Device.setProperty(XN_MODULE_PROPERTY_EMITTER_STATE, (uint8_t*)&laser_en, dataSize);</pre>

3.3.15 获取 LDM 激光开关状态

详细描述	Astra pro plus, 大白, Gemini, projector, 支持这个接口, astra 系列的产品没有获取激光状态的接口。
具体实现	<pre>int dataSize = 4; int laser_en = 0; rc = Device.getProperty(XN_MODULE_PROPERTY_EMITTER_STATE_V1, (uint8_t*)&laser_en, &dataSize);</pre>

3.3.16 设置泛光灯开关

详细描述	需要设备支持泛光灯功能, Gemini 支持泛光灯。泛光灯用在需要采集纯净 IR 的情况下。
具体实现	<pre>int dataSize = 4; int flood_en = 1; //1:表示开泛光灯;0:表示关泛光灯 openni::Status rc = Device.setProperty(XN_MODULE_PROPERTY_IRFLOOD_STATE, (uint8_t*)&flood_en, dataSize);</pre>

3.3.17 获取泛光灯开关状态

详细描述	需要设备支持泛光灯功能, Gemini 支持泛光灯。
具体实现	<pre>int flood_en = 0; int dataSize = 4; openni::Status rc = Device.getProperty(XN_MODULE_PROPERTY_IRFLOOD_STATE, (uint8_t*)&flood_en, &dataSize);</pre>

3.3.18 切换左右 IR

详细描述	切换双目设备的左右 IR, 仅对双目设备有效, 如大白, Gemini, Projector 等
具体实现	<pre>int dataSize = 4; int switch_ir = 1 或 0; //0:表示左 IR ; 1:表示右 IR openni::Status rc = Device.setProperty(XN_MODULE_PROPERTY_SWITCH_IR, (uint8_t*)&switch_ir, dataSize);</pre>

3.3.18 设置 IR FaceAEROI

详细描述	设置 usb 模组 IR ROI 坐标, 仅个别设备支持, 如海燕, 使用前请确认 Device 端固件是否支持。
具体实现	<pre>IrFaceAERoi irFaceAeroi;</pre>

```
memset(&irFaceAeroi, 0, sizeof(irFaceAeroi));
irFaceAeroi.nEnable = (uint32_t)nEnable;
irFaceAeroi.nLeft = (uint16_t)nLeft;
irFaceAeroi.nTop = (uint16_t)nTop;
irFaceAeroi.nWidth = (uint16_t)nWidth;
irFaceAeroi.nHeight = (uint16_t)nHeight;
//
openni::Status rc = Device.setProperty(XN_MODULE_PROPERTY_DEVICE_IR_FACE_AE_ROI,
&irFaceAeroi, sizeof(irFaceAeroi));
```

3.3.19 设置指定相机标定参数

详细描述	该接口用于设置不同分辨率或者比例模式下读取的相机标定参数模式，与“3.3.4”读取参数接口对应，不调用该接口设置，“3.3.4”接口默认读取 640x480 下的标定参数，该接口仅对 Dabai DW、Dabai DCW 模组开放使用，其他模组不支持。
具体实现	<pre>typedef enum { XN_CAMERA_PARAMS_640_480 = 0, XN_CAMERA_PARAMS_480_360 = 1, XN_CAMERA_PARAMS_16_9 = 2, XN_CAMERA_PARAMS_4_3 = 3, } XnCameraParamsMode; int nValue = XN_CAMERA_PARAMS_640_480 openni::Status rc = Device.setProperty(XN_MODULE_PROPERTY_CAMERA_PARAMS_MODE, &cameraParamsMode, sizeof(nValue));</pre>

4 硬件 D2C

D2C 是指的将 Depth 坐标对齐到 RGB 坐标，实现的方法有硬件 D2C 或者软件 D2C，硬件 D2C 有硬件芯片实现，不耗 CPU 资源，软件 D2C 以库的形式在软件上实现，比较耗 CPU 资源，在硬件 D2C 不满足要求的情况下使用，配置低的平台，不建议使用。

详细描述	注意： 1.设置硬件 D2C 的方法是在 Create 流之后，Start 流之前发送一下命令。设置硬件 D2C 的地方参考 6.4 设置分辨率； 2.使用硬件 D2C 接口功能前要确认设备端 Device 是否支持该功能，如果 Device 不带 RGB，硬件 D2C 是不支持的。
具体实现	<pre>typedef enum {</pre>



```
IMAGE_REGISTRATION_OFF          = 0,    //硬件 D2C 关
IMAGE_REGISTRATION_DEPTH_TO_COLOR= 1,    //硬件 D2C 开
} ImageRegistrationMode;

//开启硬件 D2C
Device.setImageRegistrationMode(openni::IMAGE_REGISTRATION_DEPTH_TO_COLOR);
//关闭硬件 D2C
Device.setImageRegistrationMode(openni::IMAGE_REGISTRATION_OFF);
```

注意：开启硬件 D2C 的命令，需要在 depth 流 Create 流后，Start 流前设置。

5 配置文件介绍

Openni.ini 放在 app 应用程序同一目录，Orbbec.ini 放在 Openni2/Drivers/ 目录。

5.1 Openni.ini

Openni.ini 配置文件（去掉前面分号开启功能）

```
[Log]
; 0 - Verbose; 1 - Info; 2 - Warning; 3 - Error. Default - None
;Verbosity=0    //输出 Log 的等级
;LogToConsole=1 //输出 Log 到控制台
;LogToFile=1    //输出 Log 到文件
;LogToAndroidLog=1 //在 Android 平台输出 log
```

5.2 Orbbec.ini

Openni.ini 配置文件

```
;----- Sensor Default Configuration -----
[Device]
; Mirroring. 0 - Off , 1 - On (default)
;设置 Device 的 mirror 属性，depth 和 ir、color 默认 mirror。
;Mirror=1

; FrameSync. 0 - Off (default), 1 - On
;设置 depth 和 color 帧同步，只有 color 是 openni 协议传输的设备支持帧同步
;FrameSync=1
```

; Stream Data Timestamps. 0 - milliseconds, 1 - microseconds (default)

;设置时间戳的单位，这个地方不用修改，用默认值。(0:表示 ms， 1: 表示微秒)

;HighResTimestamps=1

; Stream Data Timestamps Source. 0 - Firmware (default), 1 - Host

;设置时间戳源，不用修改，用默认值，用固件的时间戳 (0:固件时间戳,1:主机的时间戳)

;HostTimestamps=0

; USB interface to be used. 0 - FW Default, 1 - ISO endpoints (default on Windows), 2 - BULK endpoints (default on Linux/Mac/Android machines), 3 - ISO endpoints for low-bandwidth depth

;usb 传输类型，值为 2，不能修改

UsbInterface=2

[Depth]

; Output format. 100 - 1mm depth values (default), 102 - u9.2 Shift values.

;设置 depth 的输出格式 100: 表示 1mm 精度; 101: 100um 精度; 102: 表示输出原始视差;

;OutputFormat=100

; Is stream mirrored. 0 - Off, 1 - On

;覆盖 Device 的 Mirror 值, depth 镜像, 0: 表示关闭镜像; 1: 表示打开镜像;

;Mirror=1

; 0 - QVGA, 1 - VGA, 4 - QQVGA. Default: Arm - 4, other platforms - 0, -14 1280x720, 15 1280x960,-16 1280x800, -17 640x400 -20 320x200,-21 480x640, -25 960x1280, -26 800x1280,-27 400x640

;设置 Depth 的分辨率;

Resolution=1

; Frames per second (default is 30)

; 设置 depth 的帧率(目前不支持动态设置分辨率)

;FPS=30

; Min depth cutoff. 0-10000 mm (default is 0)

; 过滤的最小深度值，小于该深度值，过滤掉，置为 0。

; 举例：(如： MinDepthValue =300，那么小于 30 厘米的 depth 就会过滤掉)

;MinDepthValue=0

; Max depth cutoff. 0-10000 mm (default is 10000)

; 过滤最大的深度值，大于该值的 depth 会过滤掉，置为 0 。

;MaxDepthValue=10000

; Input format. 0 - Uncompressed 16-bit, 1 - PS Compression, 2 - Packed 10-bit, 3 - Packed 11-bit, 4 - Packed 12-bit. Default: Arm - 4, other platforms - 3

;深度打包格式，现在我们用到的有（10bit，11bit，12bit）其中 3：表示 11bit；4：表示 12bit；2：表示 10bit（D2 和 Atlas, Mipi 公板 设置成 2）

InputFormat=3

; software filter 0-off ,1--on (default)

;软件滤波开关，2.3.0.55 SDK 及后续版本支持

;SoftFilter=0

; Depth Rotate 0-off(default) ,1--on (only atlas device support)

;Depth 旋转功能，在 2.3.0.61 版本支持，只有 Atlas 设备有这个功能

;DepthRotate=0

; Hole Filler. 0 - Off, 1 - On (default)

; 填洞滤波，0：表示关闭滤波功能，开启时取值 1，2，3，4：(HoleFilter 代表窗口滤波器的大小，1 是 3*3，2 是 5*5，3 是 7*7，4 是 9*9)

;HoleFilter=0

[Image]

;整个[Image]字段只有对走 Openni 协议 rgb 有效，对 UVC 协议的无效；

; Output format. 200 - RGB888 (default), 201 - YUV422, 202 - Gray8 (2.0 MP only), 205 - YUYV

;设置 color 图像的输出格式，只有对 openni 输出的有效，目前支持 RGB888(200)和 YUV422(201)；

;OutputFormat=201

; Is stream mirrored. 0 - Off, 1 - On

;覆盖 Device Mirror 值，color 图像的镜像方式；0：非镜像；1：镜像

;Mirror=1

; 0 - QVGA (default), 1 - VGA, 2 - SXGA (1.3MP), 3 - UXGA (2.0MP), 14 - 720p, 15 - 1280x960

;设置 color 图像的分辨率

Resolution=1

; Frames per second (default is 30)

;设置 color 的帧率，目前我们大部分产品都是恒定帧率，不支持动态设置分辨率

;FPS=30



```
; Input format. 0 - Compressed 8-bit BAYER (1.3MP or 2.0MP only), 1 - Compressed YUV422 (default in BULK), 2 - Jpeg, 5 - Uncompressed YUV422 (default in ISO), 6 - Uncompressed 8-bit BAYER (1.3MP or 2.0MP only), 7 - Uncompressed YUYV
;设置 color 图像的打包格式，目前设置成 5,不要修改
InputFormat=5

[IR]
; Output format. 200 - RGB888, 203 - Grayscale 16-bit (default)
;设置 IR 的输出格式
;OutputFormat=203
; Is stream mirrored. 0 - Off, 1 - On
;覆盖 Device 字段的 Mirror 值，设置 IR 镜像 0：表示非镜像；1：表示镜像
;Mirror=1
; 0 - QVGA (default), 1 - VGA, 2 - SXGA(1.3MP), 4 - QQVGA, -14 1280x720, 15 1280x960,-16 1280x800, -17 640x400 -20 320x200,-21 480x640, -25 960x1280, -26 800x1280,-27 400x640
;设置 IR 分辨率，分辨率列表和 depth 一样
Resolution=1
; Frames per second (default is 30)
;设置帧率，暂时不支持动态设置：;FPS=30
```

6 配置文件 API 接口

介绍通过 API 去修改配置文件的值,例如设备的分辨率，设备不同，分辨率可能不能，通过配置文件只能配置一种分辨率，不能适用所有的设备。在 SDK 中，首先读配置文件，如果再通过下面的配置文件 API 接口，最终的参数会替换掉配置文件中设置的值。

6.1 帧同步

详细描述	只有 color 是 openni 协议的设备有帧同步功能，Color 是 UVC 协议设备不支持该功能。
具体实现	//开启帧同步，g_device 打开的设备（openni::Device） g_device.setDepthColorSyncEnabled(true); g_device.setDepthColorSyncEnabled(false);

6.2 镜像

详细描述	设置 Depth 镜像
------	-------------



具体实现	<pre>depth.setMirroringEnabled(true); //设置 depth 镜像 depth.setMirroringEnabled(false); //设置 depth 非镜像</pre>
------	---

详细描述	设置 IR 镜像
具体实现	<pre>ir.setMirroringEnabled(true); //设置 IR 镜像 ir.setMirroringEnabled(false); //设置 IR 非镜像</pre>

详细描述	设置 Color 镜像
具体实现	<pre>color.setMirroringEnabled(true); //设置 color 镜像 color.setMirroringEnabled(false); //设置 color 非镜像</pre>

6.3 获取设备的 Pid

详细描述	获取设备的 Pid
具体实现	<pre>//获取设备的 PID, g_device 打开的设备 (openni::Device) g_device.getDeviceInfo().getUSBProductId();</pre>

6.4 设置分辨率

1.设置 Depth 分辨率。

详细描述	深度输出格式(OutputFormat),需要在 openni 的 create 函数和 start 函数之间设置。 通过获取 depthVideoModes 查找支持需要的分辨率进行设置
具体实现	<pre>const Array<VideoMode>& depthVideoModes = depth.getSensorInfo().getSupportedVideoModes(); for (int i = 0; i < depthVideoModes.getSize(); i++) { VideoMode depthVideoMode = depthVideoModes[i]; if (depthVideoMode.getResolutionX() == 640 && depthVideoMode.getResolutionY() == 480 && depthVideoMode.getPixelFormat() == PIXEL_FORMAT_DEPTH_1_MM&& depthVideoMode.getFps() == 30) { depth.setVideoMode(depthVideoMode); break; } }</pre>

2. 参考源码:

源码

```
#include <stdio.h>
#include <OpenNI.h>
#include <vector>
#include <memory>
#include "OniSampleUtilities.h"
#define SAMPLE_READ_WAIT_TIMEOUT 2000 //2000ms
using namespace openni;

#if 1
int main()
{
    Status rc = OpenNI::initialize();
    if (rc != STATUS_OK)
    {
        printf("Initialize failed\n%s\n", OpenNI::getExtendedError());
        return 1;
    }

    Device device;
    rc = device.open(ANY_DEVICE);
    if (rc != STATUS_OK)
    {
        printf("Couldn't open device\n%s\n", OpenNI::getExtendedError());
        return 2;
    }

    VideoStream depth;
    if (device.getSensorInfo(SENSOR_DEPTH) != NULL)
    {
        rc = depth.create(device, SENSOR_DEPTH);
        if (rc != STATUS_OK)
        {
            printf("Couldn't create depth stream\n%s\n", OpenNI::getExtendedError());
            return 3;
        }
    }

    //Set videomode
    const Array<VideoMode>& depthVideoModes = depth.getSensorInfo().getSupportedVideoModes();
    for (int i = 0; i < depthVideoModes.getSize(); i++)
    {
        VideoMode depthVideoMode = depthVideoModes[i];
        if(depthVideoMode.getResolutionX()==640&&depthVideoMode.getResolutionY()==480&&
depthVideoMode.getPixelFormat() == PIXEL_FORMAT_DEPTH_1_MM&& depthVideoMode.getFps() == 30)
        {
            depth.setVideoMode(depthVideoMode);
            break;
        }
    }

    rc = depth.start();
    if (rc != STATUS_OK)
    {
        printf("Couldn't start the depth stream\n%s\n", OpenNI::getExtendedError());
        return 4;
    }

    VideoFrameRef frame;
    while (!wasKeyboardHit())
    {
        int changedStreamDummy;
        VideoStream* pStream = &depth;
        rc = OpenNI::waitForAnyStream(&pStream, 1, &changedStreamDummy, SAMPLE_READ_WAIT_TIMEOUT);
        if (rc != STATUS_OK)
        {
            printf("Wait failed! (timeout is %d ms)\n%s\n", SAMPLE_READ_WAIT_TIMEOUT, OpenNI::getExtendedError());
            continue;
        }

        rc = depth.readFrame(&frame);
        if (rc != STATUS_OK)
        {
            printf("Read failed!\n%s\n", OpenNI::getExtendedError());
            continue;
        }

        if (frame.getVideoMode().getPixelFormat() != PIXEL_FORMAT_DEPTH_1_MM && frame.getVideoMode().getPixelFormat() !=
PIXEL_FORMAT_DEPTH_100_UM)
        {
            printf("Unexpected frame format\n");
            continue;
        }
    }
}
```

```
DepthPixel* pDepth = (DepthPixel*)frame.getData();
int middleIndex = (frame.getHeight()+1)*frame.getWidth()/2;
printf("[%08llu] %8d\n", (long long)frame.getTimestamp(), pDepth[middleIndex]);
}
depth.stop();
depth.destroy();
device.close();
OpenNI::shutdown();
return 0;
}
#endif
```

3. IR 的设置分辨率，帧率，输出格式的方法和 Depth 一样，在 IR 流 Create 后，Start 前设置。

4. Color (openni 协议)的设置分辨率，帧率，输出格式的方法和 Depth 一样。

6.5 设置 Depth 最小值和最大值

详细描述	设置 Depth 最小值和最大值的目的是过滤掉 Depth 不再这个范围内的点的 Depth，需要在 openni 的 create 函数和 start 函数之间设置，设置的参考代码和设置 Depth 的分辨率类似。
具体实现	<pre>#include <PS1080.h> 设置 Depth 最小值: int dataSize = 4; int nMinDepth = 340; //设置 340mm openni::Status rc = depth.setProperty(XN_STREAM_PROPERTY_MIN_DEPTH, (uint8_t *)&nMinDepth, dataSize); 设置 Depth 的最大值: int dataSize = 4; int nMaxDepth =1500 ; //设置 1500mm openni::Status rc =depth.setProperty(XN_STREAM_PROPERTY_MAX_DEPTH, (uint8_t *)&nMaxDepth, dataSize);</pre>

6.6 设置 Depth 的传输格式

详细描述	需要在 openni 的 create 函数和 start 函数之间设置，设置的参考代码和设置 Depth 的分辨率类似，nInputFormaty 一般设置值 3，非必要不设置，因为 Orbbec.ini 文件配置即可。
具体实现	<pre>#include <PS1080.h> int dataSize = 4; int nInputFormat = 3; rc = depth.setProperty(XN_STREAM_PROPERTY_INPUT_FORMAT, (uint8_t *)&nInputFormat,</pre>

```
dataSize);
```

6.7 设置 Depth 软件滤波开关

详细描述	需要在 openni 的 create 函数和 start 函数之间设置, 设置的参考代码和设置 Depth 的分辨率类似。非必要不设置, 有需要在 Orbbec.ini 文件配置即可生效。
具体实现	<pre>//2:表示开启软件滤波, 0:表示关掉软件滤波 int nFilterEnable = 0; int dataSize = 4; openni::Status rc = depth.setProperty(XN_STREAM_PROPERTY_SOFTWARE_FILTER, (uint8_t *)&nFilterEnable, dataSize);</pre>

6.8 设置 Depth 旋转开关

详细描述	只有 Atlas 支持这个功能。需要在 openni 的 create 函数和 start 函数之间设置, 设置的参考代码和设置 Depth 的分辨率类似, 非必要不要设置。
具体实现	<pre>#include <PS1080.h> int dataSize = 4; int nRotateEnable = 0; //1:表示旋转, 0:表示不旋转, 只有 Atlas 有旋转 Depth 功能 openni::Status rc = depth.setProperty(XN_STREAM_PROPERTY_DEPTH_ROTATE, (uint8_t *)&nRotateEnable, dataSize);</pre>

6.9 设置 HoleFilter

详细描述	需要在 openni 的 create 函数和 start 函数之间设置, 设置的参考代码和设置 Depth 的分辨率类似。填洞滤波, 0: 表示关闭滤波功能, 开启时取值 1, 2, 3, 4 (HoleFilter 代表窗口滤波器的大小, 1 是 3*3, 2 是 5*5, 3 是 7*7, 4 是 9*9), 非必要不需要设置。
具体实现	<pre>int dataSize = 4; int nHoleFilter = 3; openni::Status rc = depth.setProperty(XN_STREAM_PROPERTY_HOLE_FILTER, (uint8_t *)&nHoleFilter, dataSize);</pre>

6.10 设置 IR 输出格式(OutputFormat)

详细描述	IR 输出格式(OutputFormat), 设置的参考代码和设置 Depth 的 OutputFormat 一样, 在设置 VideoMode
------	--

	时配置。 16bit IR: PIXEL_FORMAT_GRAY16; IR888: PIXEL_FORMAT_RGB888; IR 8bit: PIXEL_FORMAT_GRAY8; (2.3.0.67-beta2 及以后版本支持)
具体实现	<pre>VideoStream ir; if (device.getSensorInfo(SENSOR_IR) != NULL) { rc = ir.create(device, SENSOR_IR); if (rc != STATUS_OK) { printf("Couldn't create ir stream\n%s\n", OpenNI::getExtendedError()); return 3; } } const Array<VideoMode>& irVideModes = ir.getSensorInfo().getSupportedVideoModes(); for (int i = 0; i < irVideModes.getSize(); i++) { VideoMode irVideoMode = irVideModes [i]; if (irVideoMode.getResolutionX() == 640 && irVideoMode.getResolutionY() == 480 && irVideoMode.getPixelFormat() == PIXEL_FORMAT_GRAY16 && irVideoMode.getFps() == 30) { ir.setVideoMode(irVideoMode); break; } } rc = ir.start();</pre>

7 结束

《Orbbec Openni2.0-SDK C++接口使用规范》会根据 OpenNI2 SDK 的变更不断的修改和优化。